

PCS-302 : Data Structures Lab

Stack Using a Linked List | Create, Push, Pop, IsEmpty, IsFull, Size

Problem Statement

Write an algorithm and a program to implement a stack using a linked list. The program should implement the following operations: Create(), Push(k), Pop(), IsEmpty(), IsFull() and Size().

Example

```
create()    -->  Stack created, empty

push(10)    -->  Stack: 10
push(20)    -->  Stack: 10 20
push(30)    -->  Stack: 10 20 30    (top = 30)

size()      -->  3

pop()       -->  removes 30, Stack top = 20
```

Algorithm

```
Create():
    top = NULL, count = 0

Push(k):
    if IsFull(): print "Overflow", return
    create newNode with data = k
    newNode->next = top
    top = newNode
    count = count + 1

Pop():
    if IsEmpty(): print "Underflow", return
    temp = top
    top = top->next
    free temp
    count = count - 1

IsEmpty():
    return (top == NULL)

IsFull():
    try to allocate a new node
    if allocation fails: return TRUE    // no memory left
    else: free the test node, return FALSE

Size():
    return count
```

Note: a linked-list stack has no fixed capacity, so "full" only means the system has run out of memory. IsFull() checks this by attempting a test allocation.

C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;
int count = 0;

void create() {
    top = NULL;
    count = 0;
    printf("Stack created.\n");
}

int isEmpty() {
    if (top == NULL)
        return 1;
    else
        return 0;
}

int isFull() {
    struct Node *temp = (struct Node *)malloc(sizeof(struct Node));
    if (temp == NULL) {
        return 1;        // no memory left, stack is "full"
    } else {
        free(temp);
        return 0;
    }
}

void push(int k) {
    if (isFull() == 1) {
        printf("Stack Overflow. Cannot push %d.\n", k);
        return;
    }
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = k;
    newNode->next = top;
    top = newNode;
    count = count + 1;
    printf("%d pushed to stack.\n", k);
}

void pop() {
    if (isEmpty() == 1) {
        printf("Stack Underflow. Cannot pop.\n");
        return;
    }
    struct Node *temp = top;
```

```

        printf("%d popped from stack.\n", temp->data);
        top = top->next;
        free(temp);
        count = count - 1;
    }

int size() {
    return count;
}

void printStack() {
    struct Node *temp = top;
    if (isEmpty() == 1) {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack (top to bottom): ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    create();

    push(10);
    push(20);
    push(30);
    printStack();

    printf("Size = %d\n", size());

    pop();
    printStack();

    if (isEmpty() == 1)
        printf("Stack is empty.\n");
    else
        printf("Stack is not empty.\n");

    return 0;
}

```